

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU – 5741 46



MINIPROJECT REPORT - 1

ON

“Grocery Delivery Web Application using Microservices Architecture”

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN

CSE(Cyber Security)

Submitted By,

Yashraj Jangir.S

01SU23CB034

Sripad Jange

01SU23CB029

Under the guidance of
Mr. Yogesh Bandewar
IBM Trainer

2025-2026

ER

DEPARTMENT OF CYBER SECURITY
SRINIVAS UNIVERSITY, MUKKA

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU – 5741 46



DEPARTMENT OF CYBER SECURITY

CERTIFICATE

This is to certify that the miniproject entitled ***“Grocery Delivery Web Application using Microservices Architecture”*** is a bonafide work carried out by ***Yashraj Jangir.S, Sripad Jange*** bearing the USN ***01SU23CB034, 01SU23CB034*** in the partial fulfillment for the award of **Bachelor of Technology** in CSE(Cyber Security) of the **Srinivas University, Institute of Engineering & Technology** during the year **2025-2026**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The mini project report has been approved as it satisfies the academic requirements in respect of internship work prescribed for the said degree.

Mr. Yogesh Bandewar
IBM Trainer (Sri Thulasi EduTech Pvt.Ltd)
(Miniproject Guide)

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU – 5741 46



DEPARTMENT OF CYBER SECURITY

DECLARATION

We, **Yashraj Jangir.S, Sripad Jange**, the student of fifth semester, **B.Tech** in CSE(Cyber Security), Srinivas University, Mukka, hereby declare that the mini project entitled ***"Grocery Delivery Web Application using Microservices Architecture"*** has been successfully completed by us in partial fulfillment of the requirements for the award of degree in **Bachelor of Technology in Computer Science and Engineering of Srinivas University, Institute of Engineering and Technology** and no part of it has been submitted for the award of degree or diploma in any university or institution previously.

Date: _____

Place: Mukka

ABSTRACT

This project presents the design and development of a full-stack Grocery Delivery Web Application built using a modern microservices architecture. The system is designed to simulate real-world online grocery platforms such as Blinkit and Zepto, providing users with a convenient, efficient, and user-friendly way to purchase essential items. The application allows users to browse a wide range of products, manage their cart, place orders, and track deliveries in real-time through a responsive web interface.

The frontend of the application is developed using React (Vite), which ensures fast loading speed, smooth navigation, and an interactive user experience. The user interface is designed to be intuitive and responsive, enabling seamless interaction across different devices. On the backend, the system is implemented using Node.js and Express.js, following a microservices architecture where the application is divided into multiple independent services. These include the API Gateway, Product Service, Cart Service, Order Service, Delivery Service, and Notification Service. Each service is responsible for a specific functionality and communicates with others through REST APIs, ensuring a loosely coupled and modular system design.

The system supports core functionalities such as dynamic product listing, real-time cart management, efficient order processing, and delivery tracking. Users can add, update, and remove items from their cart, place orders with ease, and receive updates regarding their order status. The delivery module ensures transparency by tracking the progress of orders through different stages, while the notification service keeps users informed with timely updates.

The adoption of microservices architecture provides several advantages over traditional monolithic systems. It enhances scalability by allowing individual services to be scaled independently based on demand, improves fault tolerance by isolating failures within specific services, and simplifies maintenance by enabling updates without affecting the entire system. This architecture also supports flexibility, making it easier to integrate new features and technologies in the future.

In addition to functional capabilities, the project demonstrates key software engineering principles such as distributed system design, separation of concerns, modular development, and API-based communication. The integration of frontend and backend components is carefully managed to ensure efficient data flow and a smooth user experience.

Although the current implementation focuses on essential features, the system is designed with future scalability in mind. It can be extended to include advanced functionalities such as secure payment gateway integration, real-time communication using WebSockets, cloud deployment for global accessibility, and mobile application development. Furthermore, performance optimization techniques such as caching and load balancing can be incorporated to enhance system efficiency under high user traffic.

Overall, this project serves as a comprehensive academic prototype and a portfolio-level implementation of a modern grocery delivery platform. It effectively demonstrates how contemporary web technologies and architectural patterns can be combined to build scalable, reliable, and user-centric applications that align with current industry standards.

ACKNOWLEDGEMENT

It is our pleasure to acknowledge for all those who have provided guidance, inspiration and encouragement for the successful completion of my mini project.

We take this opportunity to express our profound gratitude to our respected **Mr. Yogesh Bandewar** IBM Trainer (Sri Thulasi EduTech Pvt.Ltd)for his ever inspiring guidance, constant encouragement and support.

- **Yashraj Jangir.S**
- **Sripad Jange**

TABLE OF CONTENTS

CHAPTER	TITLE	Page No.
CHAPTER 1	INTRODUCTION	1
1.1	Objectives	2
1.2	Overview	2
1.3	Problem Statement	3
1.4	Solution	3
CHAPTER 2	LITERATURE SURVEY	4
CHAPTER 3	REQUIREMENT SPECIFICATIONS	5
CHAPTER 4	SYSTEM DESIGN	6
4.1	Project Modules	6
4.2	System Architecture	6
4.3	System Workflow	7
CHAPTER 5	MODULES OF THE SYSTEM	8
CHAPTER 6	TESTING	9
CHAPTER 7	IMPLEMENTATION	10
CHAPTER 8	RESULTS	11
8.1	System Overview Interface	11
8.2	Prediction Result – BIG HIT	12
8.3	Prediction Result – HIT	12
CHAPTER 9	CHALLENGES AND LIMITATIONS	13
CHAPTER 10	FUTURE ENHANCEMENTS	14
CHAPTER 11	CONCLUSION	15
CHAPTER 12	UML DIAGRAMS	16
12.1	Use Case Diagram	16
12.2	Class Diagram	17
12.3	Sequence Diagram	17
12.4	Collaboration Diagram	19
CHAPTER 13	REFERENCES	20

LIST OF FIGURES

FIGURE NUMBER	TITLE	PAGE NO.
4.2.1	System Architecture	6
8.1.1	System Overview Interface	11
8.2.1	Cart and Order Management	12
8.3.1	Order Tracking	12
12.1.1	Use Case Diagram - Grocery Delivery System	16
12.2.1	Class Diagram – Grocery Delivery System	17
12.3.1	Sequence Diagram – Grocery Delivery System	18
12.4.1	Collaoration Diagram – Grocery Delivery System	19

Chapter 1

INTRODUCTION

The rapid evolution of digital technologies and internet penetration has transformed the way people purchase everyday essentials. Among these, grocery delivery services have emerged as one of the most impactful innovations in modern e-commerce. Platforms such as Blinkit, Zepto, and Swiggy Instamart have revolutionized traditional shopping by offering instant delivery of groceries, eliminating the need for physical store visits and saving valuable time for users.

In today's fast-paced lifestyle, users demand convenience, speed, and reliability. Traditional grocery shopping methods often involve long queues, limited product availability, and time constraints. To address these challenges, online grocery delivery systems have become increasingly popular, providing users with the ability to browse products, add items to a cart, place orders, and track deliveries—all from the comfort of their homes.

This mini-project focuses on the design and development of a **Grocery Delivery Web Application using Microservices Architecture**. The system is built using modern web technologies, with a frontend developed in React (Vite) and a backend structured into multiple independent microservices using Node.js and Express.

Unlike monolithic architectures, where all functionalities are tightly coupled into a single application, this project adopts a **microservices-based approach**, where each service is independently developed, deployed, and maintained. The application is divided into multiple services such as Product Service, Cart Service, Order Service, Delivery Service, Notification Service, and an API Gateway that acts as a central entry point.

This architectural approach enhances scalability, flexibility, and fault isolation. For example, if the cart service fails, it does not affect other services like product browsing or delivery tracking. Each service can also be scaled independently based on demand, making the system more efficient and production-ready.

1.1 OBJECTIVES

- To design and develop a full-stack grocery delivery web application
- To implement microservices architecture for better scalability and modularity
- To create an interactive and responsive user interface using React
- To enable efficient cart management and order processing
- To implement real-time delivery tracking functionality
- To ensure seamless communication between multiple backend services
- To demonstrate modern web development and distributed system concepts
- To build a portfolio-level project aligned with industry standards

1.2 Overview

The Grocery Delivery Web Application is a full-stack system that enables users to perform all essential online shopping activities, including browsing products, managing carts, placing orders, and tracking deliveries.

The system is divided into two major components:

1. Frontend

Developed using React (Vite), the frontend provides an intuitive and responsive interface for users. It includes pages such as product listing, cart management, and order tracking.

2. Backend (Microservices)

The backend is composed of multiple independent services, each responsible for a specific functionality:

- **API Gateway:** Handles all incoming requests and routes them to appropriate services
- **Product Service:** Manages product data and listings
- **Cart Service:** Handles cart operations like adding and removing items
- **Order Service:** Processes and manages user orders
- **Delivery Service:** Tracks and updates delivery status
- **Notification Service:** Sends updates to users

These services communicate via REST APIs, ensuring modular and scalable system design.

1.3 Problem Statement

In today's rapidly evolving digital world, the demand for online grocery delivery systems has significantly increased due to changing consumer lifestyles and the need for convenience. Traditional grocery shopping methods often require users to physically visit stores, spend time selecting products, wait in queues, and manage transportation of goods. These limitations make the process inefficient, especially for working professionals, students, and elderly individuals.

Although several online grocery platforms exist, many systems are built using monolithic architectures where all functionalities—such as product management, cart handling, order processing, and delivery tracking—are tightly coupled within a single application. This creates multiple challenges:

Scalability Issues: As the number of users increases, the system struggles to handle high traffic efficiently. Scaling a monolithic system requires scaling the entire application, which is resource-intensive and inefficient.

Poor Maintainability: Any change or update in one part of the system may affect other components, making debugging and maintenance complex and time-consuming.

Single Point of Failure: If one module fails, the entire system may become unavailable, affecting the overall user experience.

Limited Flexibility: It becomes difficult to introduce new features or technologies due to tightly coupled components.

Performance Bottlenecks: High load on specific modules (such as cart or order processing) can slow down the entire system.

Lack of Real-Time Updates: Many systems fail to provide accurate real-time order tracking and notifications, leading to poor user satisfaction.

1.4 Solution

To overcome the limitations of traditional and monolithic grocery delivery systems, this project proposes a microservices-based Grocery Delivery Web Application. The system is designed by dividing the backend into multiple independent services such as Product Service, Cart Service, Order Service, Delivery Service, Notification Service, and an API Gateway that manages all client requests.

Each service performs a specific function and communicates with others through REST APIs. This modular approach improves scalability, as individual services can be scaled based on demand without affecting the entire system. It also enhances maintainability, allowing developers to update or modify one service without impacting others.

Chapter 2

LITERATURE SURVEY

The growth of e-commerce and digital technologies has led to the rapid development of online grocery delivery systems. These platforms aim to provide convenience, efficiency, and time-saving solutions by allowing users to purchase essential items from their homes. Over time, various technologies and system architectures have been explored to improve their performance and user experience.

Earlier systems were developed using **monolithic architecture**, where all functionalities such as product management, cart operations, and order processing were combined into a single application. While this approach was simple, it created challenges such as poor scalability, difficulty in maintenance, and higher risk of system failure. As user demand increased, these limitations became more evident.

To address these issues, **microservices architecture** emerged as a modern solution. In this approach, the application is divided into multiple independent services, each handling a specific function. This improves scalability, as individual services can be scaled independently based on demand. It also enhances flexibility and fault tolerance, ensuring that failure in one service does not affect the entire system. Many large-scale platforms have adopted this architecture due to its efficiency and reliability.

In addition to backend architecture, frontend technologies have also evolved significantly. Frameworks like **React** are widely used to build fast and responsive user interfaces. React's component-based structure enables efficient rendering and improves user interaction, making it suitable for modern web applications.

The use of **RESTful APIs** plays a crucial role in connecting frontend and backend services. These APIs enable smooth communication between different components of the system and ensure structured data exchange. This approach is essential in microservices-based systems where multiple services need to interact seamlessly.

Furthermore, modern applications focus on providing **real-time features** such as order tracking and notifications. These features enhance user experience by offering transparency and timely updates about order status.

In conclusion, existing studies highlight the importance of using scalable architectures, modern frontend frameworks, and efficient communication mechanisms to build reliable grocery delivery systems.

Chapter 3

REQUIREMENT SPECIFICATIONS

Requirement specification is a detailed description of the system requirements needed to develop and run the Grocery Delivery Web Application. It includes both hardware and software requirements that ensure smooth functioning, performance, and reliability of the system.

Hardware Requirements

- **Processor:** Intel i5 or higher
- **RAM:** Minimum 8 GB
- **Storage:** 512 GB SSD recommended
- **Display:** Full HD (1920 × 1080) for better UI experience
- **Internet Connection:** Stable broadband connection for API communication

Software Requirements

- **Operating System:** Windows 10/11 or Linux
- **Frontend:** React (Vite)
- **Backend:** Node.js with Express.js
- **Database:** MongoDB

Development Tools and Technologies

- **React (Vite):** Used for building a fast and responsive user interface
- **Node.js:** Provides server-side runtime environment
- **Express.js:** Framework for building REST APIs
- **REST APIs:** Used for communication between frontend and backend services
- **Microservices Architecture:** Enables modular and scalable backend design

IDE and Tools

- **IDE:** Visual Studio Code (VS Code)
- **Browser:** Google Chrome / Edge
- **Version Control:** Git & GitHub

Chapter 4

SYSTEM DESIGN

4.1 Project Modules

- **API Gateway:** Acts as the main entry point for all client requests and routes them to appropriate services.
- **Product Service:** Manages product details such as name, category, and price, allowing users to browse items.
- **Cart Service:** Handles cart operations like adding, updating, and removing products.
- **Order Service:** Processes user orders and maintains order records.
- **Delivery Service:** Tracks and updates the delivery status of orders.
- **Analysis Generator:** Produces a detailed textual report with insights

4.2 System Architecture

The system follows a modular architecture:

- **Presentation Layer:** React-based frontend for user interaction
- **API Gateway Layer:** Handles request routing and communication
- **Service Layer:** Includes Product, Cart, Order, Delivery, and Notification services

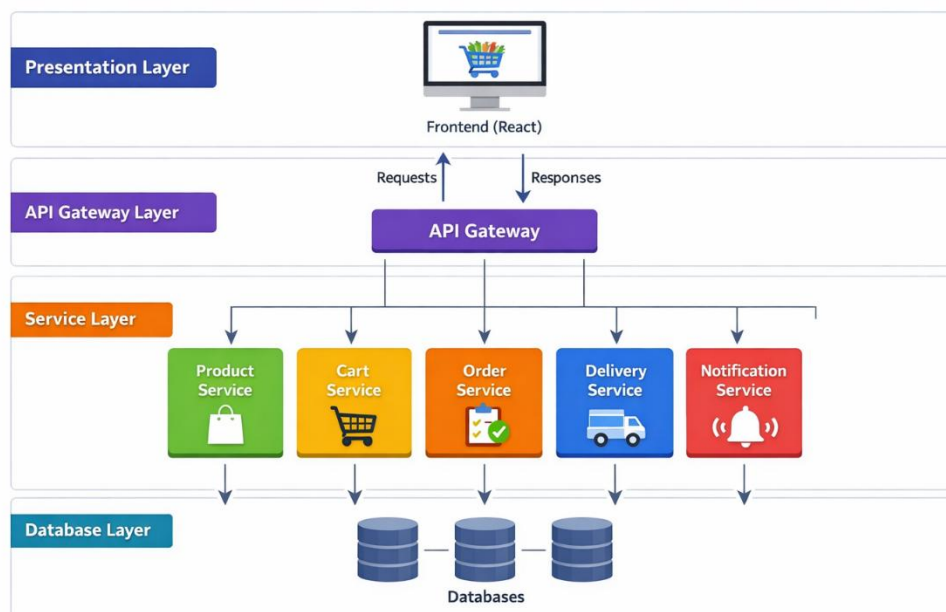


Figure 4.2.1 System Architecture

4.3 SYSTEM WORKFLOW

The system workflow describes the step-by-step process of how the application functions from user interaction to final output.

Step 1: User Interaction

The user accesses the application through the frontend and browses available grocery products.

Step 2: Request Handling

User actions such as viewing products or adding items to the cart are sent to the API Gateway.

Step 3: Service Processing

The API Gateway forwards requests to the respective microservices:

- Product Service handles product data
- Cart Service manages cart operations
- Order Service processes orders

Step 4: Order Placement

When the user places an order, the Order Service processes the request and stores order details in the database.

Step 5: Delivery Management

The Delivery Service updates the delivery status and tracks the order progress.

Step 6: Notification

The Notification Service sends real-time updates to the user regarding order status.

Step 7: Response to User

The processed data is sent back to the frontend through the API Gateway, and the user interface is updated accordingly.

Chapter 5

MODULES OF THE SYSTEM

Modules and Their Descriptions:

1.API Gateway

- Acts as the central entry point for all client requests
- Routes requests to appropriate backend services
- Handles communication between frontend and microservices
- Provides a unified interface for the system

2. Product Service

- Manages product data such as name, category, and price
- Provides product listings to users
- Supports browsing and viewing of grocery items
- Ensures availability and accuracy of product information

3. Cart Service

- Allows users to add products to the cart
- Updates product quantities and removes items
- Maintains cart state for each user
- Ensures smooth cart operations

4. Order Service

- Processes user orders and stores order details
- Calculates total cost of items in the cart
- Maintains order history
- Ensures proper order management

5. Delivery Service

- Tracks delivery status of orders
- Updates delivery stages (placed, out for delivery, delivered)
- Provides real-time tracking information
- Ensures smooth delivery flow

Chapter 6

TESTING

To ensure the reliability and usability of the Grocery Delivery Web Application, testing was conducted across three key dimensions: functionality, user interface, and system performance. Each module of the application was tested carefully to ensure smooth operation and accurate results.

1. Functional Testing

The core functionalities of the application were validated through input checks and output verification. Product listing was tested to ensure correct display of items with accurate details such as name, price, and category. Cart operations, including adding items, updating quantities, and removing products, were verified for correctness. Order placement functionality was tested to ensure proper processing and storage of order details. Delivery status updates were checked to confirm accurate tracking of order stages. The notification system was also tested to ensure users receive timely updates regarding their orders.

2. UI Testing

The user interface was tested for responsiveness and usability across different screen sizes. Layout, alignment, and spacing of components were reviewed to ensure a clean and consistent design. Buttons, input fields, and navigation elements were tested for proper functionality and responsiveness. Real-time updates in the cart and order tracking sections were verified for smooth user interaction. Overall user experience was evaluated to ensure ease of use and intuitive navigation throughout the application.

3. System Testing

System performance and integration were evaluated by testing the complete workflow of the application. End-to-end processes such as product browsing, cart management, order placement, and delivery tracking were tested to ensure seamless interaction between modules. Communication between microservices through the API Gateway was verified for accuracy and efficiency. The system's behavior under multiple user requests was observed to ensure stability and reliability. Error handling mechanisms were also tested to confirm proper responses in case of failures.

Chapter 7

IMPLEMENTATION

The system is implemented using modern web development technologies to ensure modularity, scalability, and ease of maintenance. The application follows a microservices architecture, where the backend is divided into independent services such as API Gateway, Product Service, Cart Service, Order Service, Delivery Service, and Notification Service. Each service is responsible for a specific functionality and communicates with others through REST APIs, ensuring a clean and loosely coupled system design.

The frontend is developed using React (Vite), providing a fast and responsive user interface. The UI is designed with a focus on usability and smooth interaction, allowing users to browse products, manage their cart, and place orders easily. Components are structured in a reusable manner, ensuring consistency across the application. Dynamic updates, such as cart changes and order status, are reflected instantly to enhance user experience.

The backend is built using Node.js and Express.js, where each microservice handles its own logic and data. The API Gateway acts as a central point that receives requests from the frontend and routes them to the appropriate services. Each service processes the request and returns the response through the gateway, ensuring efficient communication and data flow across the system.

The Product Service manages product-related data, while the Cart Service handles operations such as adding and removing items. The Order Service processes orders and stores order details, and the Delivery Service tracks order progress and updates delivery status. The Notification Service ensures that users receive timely updates about their orders, improving transparency and engagement.

Data management is handled using a database system where each service interacts with its respective data. Proper API design ensures smooth integration between services and maintains consistency across the application. Error handling mechanisms are implemented to manage failures and provide appropriate responses to users.

Overall, the implementation combines a modern frontend with a scalable microservices backend, resulting in a robust and efficient grocery delivery system. The system demonstrates real-world application development practices and provides a strong foundation for further enhancements and deployment.

Chapter 8

RESULTS

The developed Grocery Delivery Web Application demonstrates efficient performance with a responsive user interface and smooth communication between microservices. The system successfully enables users to browse products, manage carts, place orders, and track delivery status. The modular architecture ensures reliable performance and seamless interaction between different components.

Figure 8.1: System Overview

The system overview interface represents the main homepage of the application where users can browse available grocery products. It displays product details such as name, price, and category along with options to add items to the cart. The interface is designed to be simple, responsive, and user-friendly.

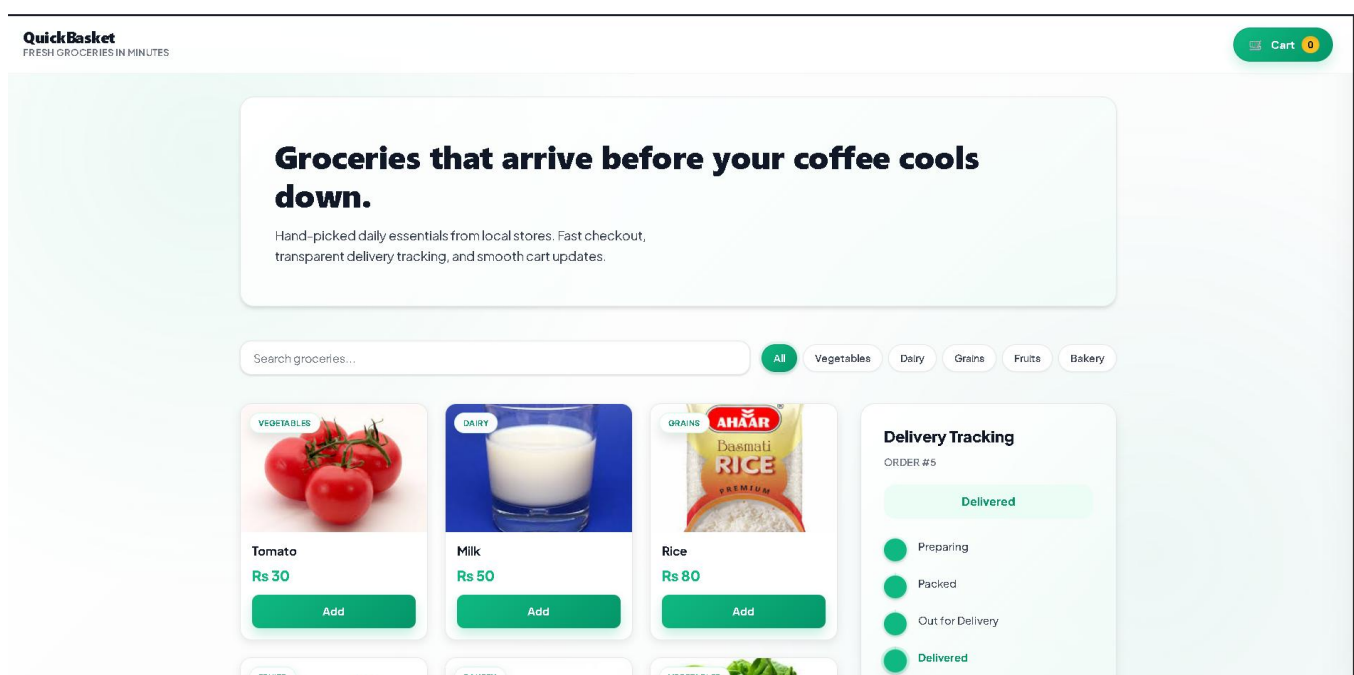


Figure 8.1.1 : System Overview Interface

Figure 8.2: Cart and Order Management

This section shows the cart and order functionality of the application. Users can add products to the cart, update quantities, and remove items. The system calculates the total cost dynamically and allows users to place orders easily.

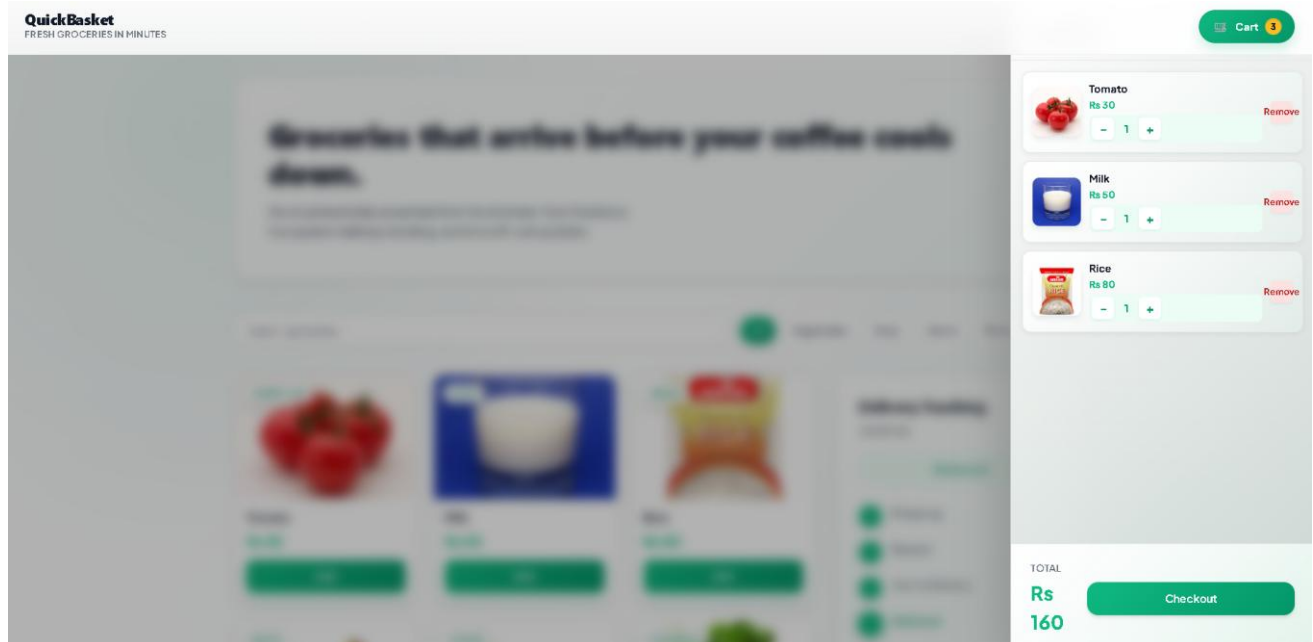


Figure 8.2.1: Cart and Order Management

Figure 8.3: Order Tracking

The order tracking interface allows users to view the status of their orders. It displays different stages such as order placed, out for delivery, and delivered, providing transparency and real-time updates.

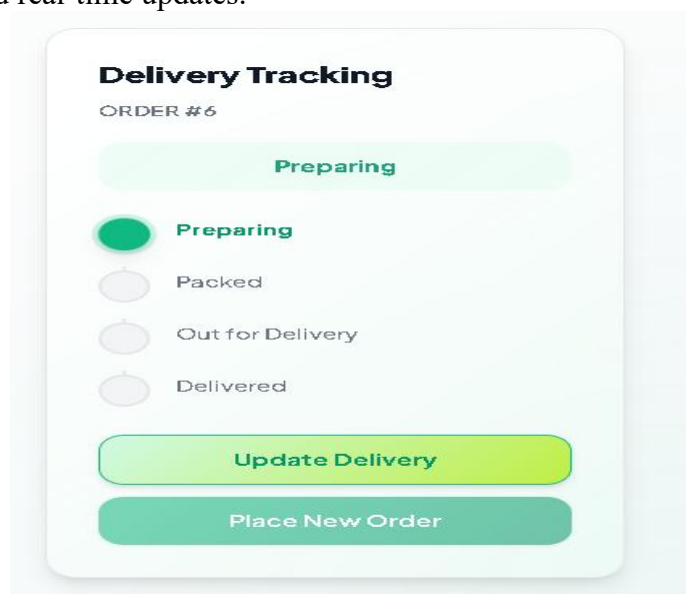


Figure 8.3.1 : Order Tracking

Chapter 9

CHALLENGES AND LIMITATIONS

While the system demonstrates effective functionality and a scalable microservices design, several challenges and limitations were encountered during development:

- **Microservices Complexity:** Managing multiple services increased the complexity of development, debugging, and integration compared to a monolithic system. Proper coordination between services was required to ensure smooth communication.
- **Inter-Service Communication Overhead:** Communication between microservices through APIs introduced latency and required careful handling of request-response cycles, especially during high traffic.
- **Limited Authentication and Security:** The current system does not include advanced authentication and authorization mechanisms, which may affect data security in real-world deployment.
- **No Payment Integration:** The application does not include an online payment gateway, limiting it to order simulation rather than complete real-world functionality.
- **Basic Error Handling:** Although basic error handling is implemented, advanced mechanisms such as centralized logging and monitoring are not included.
- **Scalability Not Fully Tested:** While the system is designed to be scalable, it has not been tested under heavy real-world traffic conditions.
- **UI Limitations:** The user interface is functional but may lack advanced design features and animations found in production-level applications.

Despite these limitations, the system serves as a strong academic project, demonstrating the practical implementation of microservices architecture and modern web development techniques. It provides a solid foundation for further enhancements and real-world deployment.

Chapter 10

FUTURE ENHANCEMENTS

To extend the capabilities and real-world applicability of the system, several enhancements are proposed for future development:

- **Payment Gateway Integration:** Integrate secure payment systems such as Razorpay or Stripe to enable online transactions, allowing users to complete purchases seamlessly within the platform.
- **User Authentication and Authorization:** Implement secure login and registration using technologies like JWT or OAuth. This will ensure data security, user-specific data handling, and personalized user experiences.
- **Real-Time Tracking with WebSockets:** Enhance delivery tracking by implementing real-time communication using WebSockets, enabling instant updates for order status without requiring page refresh.
- **Admin Dashboard:** Develop a comprehensive admin panel to manage products, orders, users, and delivery operations. This dashboard can include analytics such as total orders, revenue, and product performance.
- **Mobile Application Development:** Extend the system to mobile platforms using frameworks like React Native or Flutter, making the application accessible to a wider audience.
- **Advanced UI/UX Improvements:** Improve the frontend design by adding modern animations, better layouts, and enhanced visual elements to match industry-level applications and improve user engagement.
- **Caching and Performance Optimization:** Implement caching mechanisms such as Redis to reduce API response time and improve system performance under high load conditions.
- **Cloud Deployment and Scalability:** Deploy the system on cloud platforms like AWS, Azure, or Vercel to ensure high availability, scalability, and better performance across different regions.
- **AI-Based Recommendations:** Introduce recommendation systems that suggest products to users based on their previous purchases and browsing behavior, enhancing user experience and engagement.

These enhancements aim to transform the current academic project into a robust, scalable, and production-ready grocery delivery platform that aligns with modern industry standards.

Chapter 11

CONCLUSION

This project successfully demonstrates the development of a scalable and efficient Grocery Delivery Web Application using microservices architecture. By combining modern frontend technologies with a distributed backend system, it delivers a practical and user-friendly platform that allows users to browse products, manage carts, place orders, and track deliveries in real-time. The system is modular, extensible, and visually responsive—making it well-suited for academic presentation, portfolio showcase, and further enhancement into a production-level application.

The integration of multiple independent services such as Product, Cart, Order, Delivery, and Notification through an API Gateway highlights how complex systems can be structured in a simplified and maintainable manner. This approach ensures better scalability, fault tolerance, and flexibility compared to traditional monolithic systems. The use of React for frontend development enables a smooth and interactive user experience, while Node.js and Express provide an efficient and scalable backend environment for handling requests and managing data.

Throughout the development process, the project emphasizes key principles of modern software engineering:

- Modular architecture to improve scalability and maintainability
- API-based communication for efficient interaction between services
- Separation of concerns for better system organization
- Responsive and user-friendly interface design for enhanced user experience

The project also demonstrates how real-world applications can benefit from distributed system design. Each service operates independently, allowing easy updates, faster debugging, and improved system stability. This makes the application adaptable to future technological advancements and feature additions.

Although the current implementation focuses on core functionalities, it provides a strong foundation for further enhancements such as secure payment integration, real-time communication, cloud deployment, and mobile application support. With these improvements, the system can evolve into a fully functional, industry-ready grocery delivery platform.

Overall, this project serves as a practical example of applying microservices architecture and modern web development technologies to solve real-world problems. It reflects both technical understanding and implementation capability, making it a valuable addition to academic and professional portfolios.

Chapter 12

UML DIAGRAMS

This chapter presents the Unified Modeling Language (UML) diagrams for the Grocery Delivery Web Application. UML diagrams are used to visually represent the system from different perspectives, including user interactions, system structure, and communication between components. These diagrams help in understanding the design and functionality of the system in a clear and structured manner. The key diagrams included in this chapter are Use Case Diagram, Class Diagram, Sequence Diagram, and Collaboration Diagram.

12.1 Use Case Diagram

The Use Case Diagram illustrates the interaction between users and the system. The primary actor is the user, who can perform actions such as browsing products, adding items to the cart, placing orders, and tracking delivery status. An additional actor, the admin, is responsible for managing products, updating details, and monitoring system operations.

The diagram shows how users interact with different functionalities of the system and highlights the main use cases supported by the application. It provides a high-level overview of system behavior from the user's perspective.

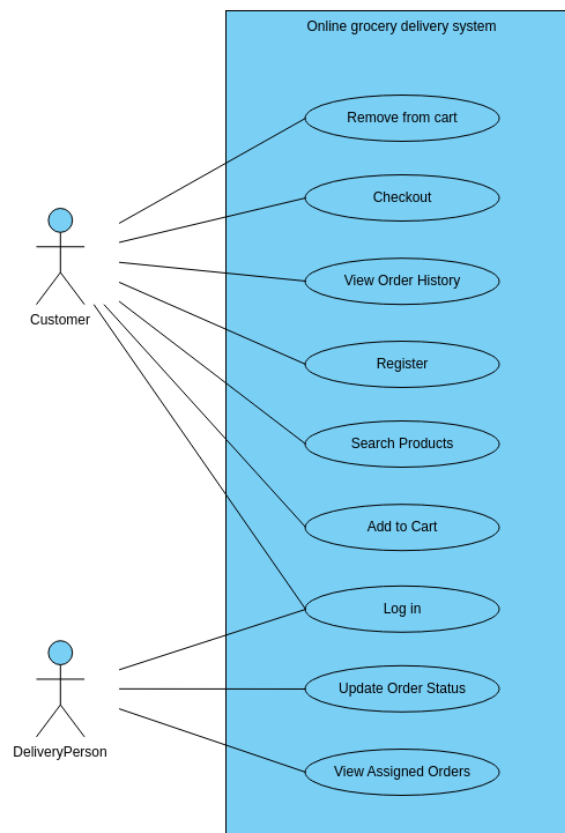


Figure 12.1: Use Case Diagram — Grocery Delivery System

12.2 Class Diagram

The Class Diagram represents the structural design of the system. It includes important classes such as User, Product, Cart, Order, and Delivery, along with their attributes and relationships. Each class defines specific properties and methods that contribute to system functionality.

In addition to domain classes, the diagram also represents service-level components such as API Gateway and various microservices. These components show how responsibilities are distributed across the system. The relationships between classes, such as associations and dependencies, help in understanding how data flows within the application.

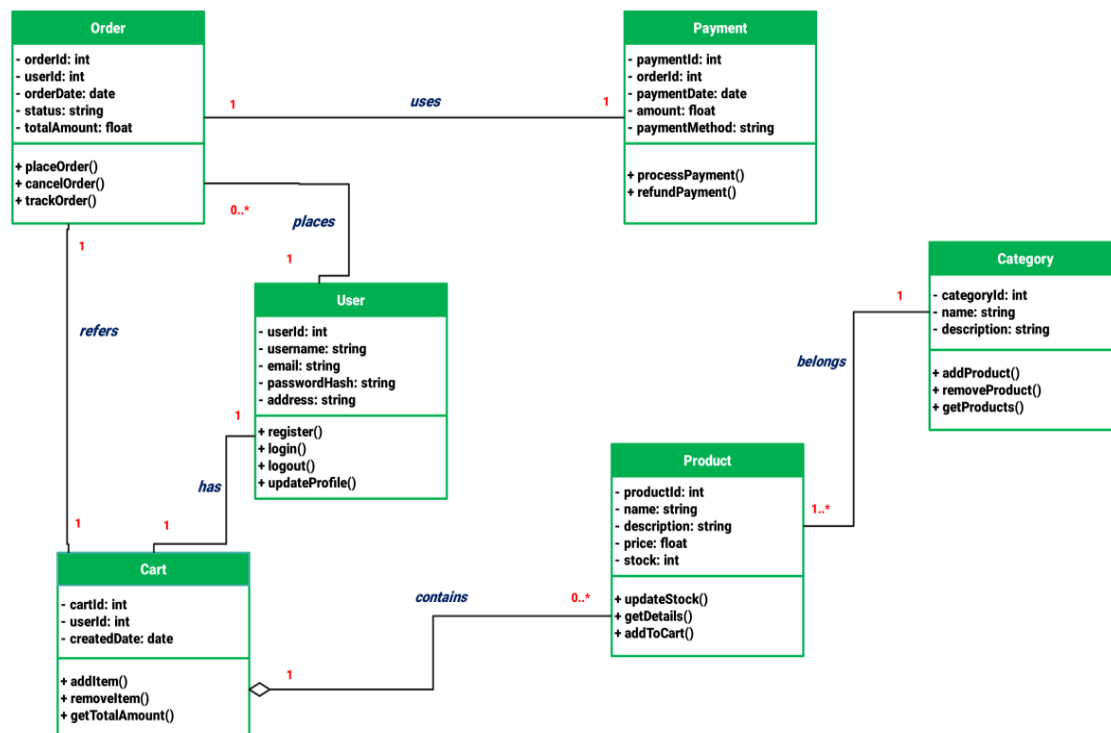


Figure 12.2: Class Diagram — Grocery Delivery System

12.3 Sequence Diagram

The Sequence Diagram describes the flow of operations in a time-based sequence. It shows how different components interact during processes such as placing an order. The diagram includes entities such as the user, frontend interface, API Gateway, and backend services.

The interaction begins when the user performs an action, such as placing an order. The request is sent from the frontend to the API Gateway, which forwards it to the appropriate microservices. Responses are then returned step-by-step until the final result is displayed to the user. This diagram clearly represents the order of execution and communication flow within the system.

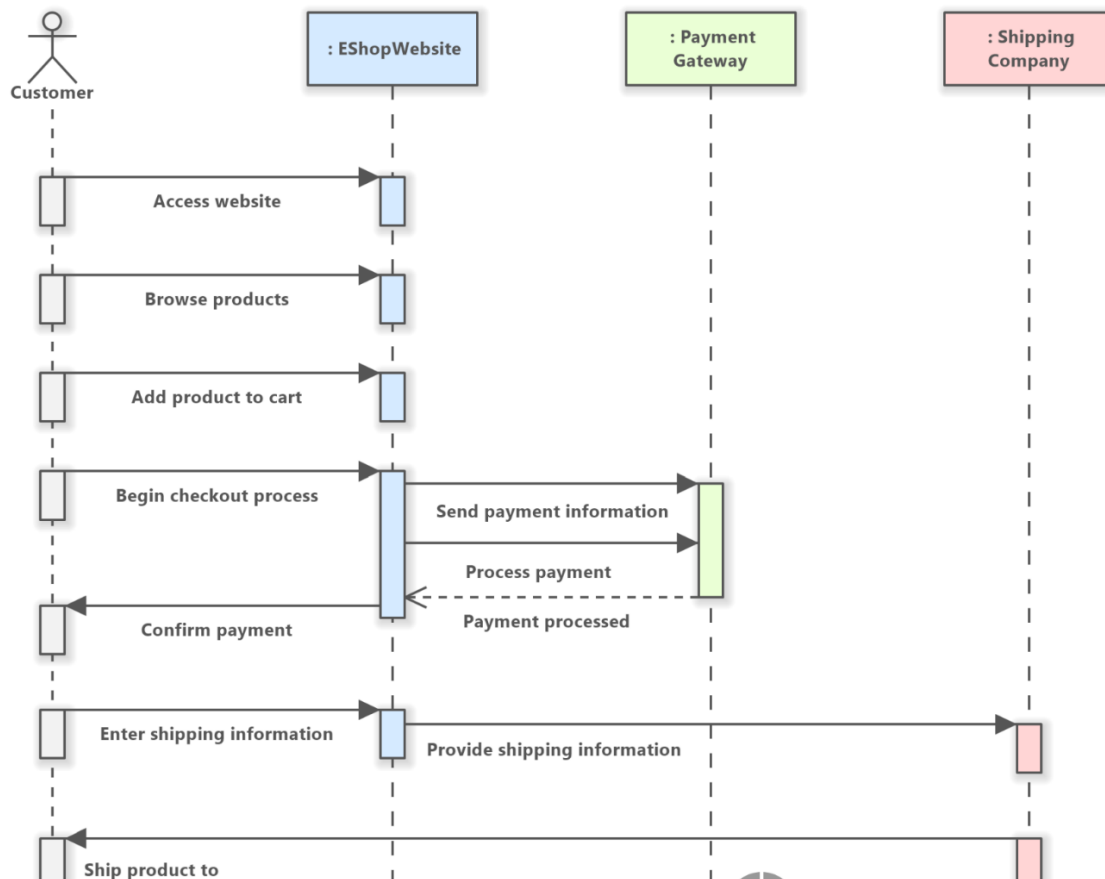


Figure 12.3: Sequence Diagram — Order Processing Flow

12.4 Collaboration Diagram

The Collaboration Diagram (also known as Communication Diagram) shows how different components of the system interact with each other. Unlike the Sequence Diagram, it focuses more on the relationships between objects rather than the time sequence.

The diagram highlights how the frontend communicates with the API Gateway and how the gateway interacts with different microservices such as Product, Cart, Order, and Delivery services. It provides a clear view of how data and requests move across the system and how components collaborate to complete a task.

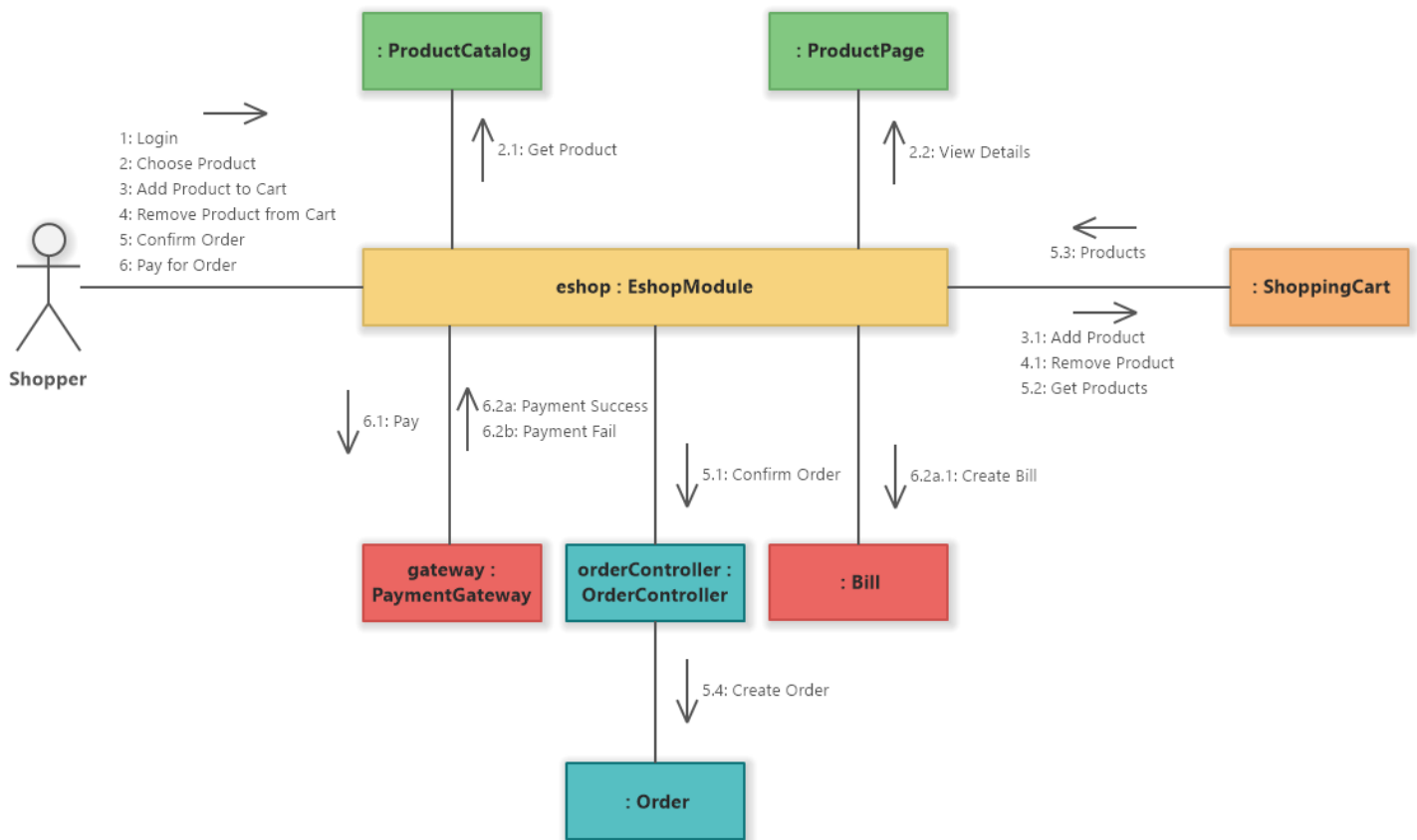


Figure 12.4: Collaboration Diagram — System Communication

Chapter 13

REFERENCE

- React Documentation. (2026). React – A JavaScript library for building user interfaces. Retrieved from <https://react.dev>
- • Node.js Documentation. (2026). Node.js Official Documentation. Retrieved from <https://nodejs.org>
- • Express.js Documentation. (2026). Fast, unopinionated, minimalist web framework for Node.js. Retrieved from <https://expressjs.com>
- • MongoDB Documentation. (2026). MongoDB Database Documentation. Retrieved from <https://www.mongodb.com>
- • Richardson, C. (2018). Microservices Patterns: With examples in Java. Manning Publications.
- • Newman, S. (2015). Building Microservices. O'Reilly Media.
- • REST API Tutorial. (2026). RESTful Web Services Concepts. Retrieved from <https://restfulapi.net>
- • AWS Documentation. (2026). Cloud Computing Services. Retrieved from <https://aws.amazon.com>